

Nested Cross-Validation

2026-04-17

Introduction

When hyperparameters are tuned on the same data used for evaluation, performance estimates are optimistically biased — the chosen hyperparameters fit not only the underlying signal but also the noise in the validation folds, producing CV errors lower than the model achieves on truly new data. Nested cross-validation separates the two concerns: an inner loop tunes hyperparameters on each training fold, and an outer loop evaluates the resulting tuned model on held-out test folds. The outer-loop mean is an honest, low-bias estimate of generalisation error.

Prerequisites

A working understanding of k -fold cross-validation, hyperparameter tuning, and the optimism bias that arises from tuning and evaluating on the same data.

Theory

The standard nested-CV scheme:

- **Outer loop:** K -fold CV. Each fold splits data into outer-training and outer-test sets.
- **Inner loop:** on each outer-training set, run K' -fold CV to pick the best hyperparameter configuration.
- **Outer evaluation:** refit the chosen configuration on the full outer-training set; predict and score on the outer-test set.
- **Aggregate:** average the K outer-test scores; the mean is the generalisation estimate, and the spread quantifies uncertainty.

Typical values: outer $K = 5$ to 10 , inner $K' = 5$. Computational cost scales as $K \times K'$ times the cost of fitting one model.

Assumptions

Observations are iid (or the resampling respects the dependence structure for time-series, clustered, or grouped data); all preprocessing (imputation, scaling, feature engineering) is refit within each inner fold to prevent test-set leakage.

R Implementation

```
library(rsample); library(purrr); library(dplyr); library(tidyr)

set.seed(2026)
n <- 300
df <- tibble(x1 = rnorm(n), x2 = rnorm(n),
             y = 1 + 2 * rnorm(n) + 0.5 * rnorm(n)^2)

outer_folds <- vfold_cv(df, v = 5)
lambdas <- c(0.001, 0.01, 0.1, 1, 10)
```

```

fit_ridge <- function(train, lambda) {
  X <- model.matrix(y ~ x1 + x2, train)[, -1]
  solve(crossprod(X) + lambda * diag(ncol(X))) %*% crossprod(X, train$y)
}

pred_ridge <- function(beta, newdata) {
  as.vector(model.matrix(y ~ x1 + x2, newdata)[, -1] %*% beta)
}

outer_rmse <- map_dbl(outer_folds$plits, function(osp) {
  tr_o <- analysis(osp); te_o <- assessment(osp)

  inner <- vfold_cv(tr_o, v = 5)
  inner_rmse <- map_dfr(lambdas, function(l) {
    errs <- map_dbl(inner$plits, function(isp) {
      b <- fit_ridge(analysis(isp), l)
      sqrt(mean((assessment(isp)$y - pred_ridge(b, assessment(isp)))^2))
    })
    tibble(lambda = l, rmse = mean(errs))
  })
  best_l <- inner_rmse$lambda[which.min(inner_rmse$rmse)]
  b <- fit_ridge(tr_o, best_l)
  sqrt(mean((te_o$y - pred_ridge(b, te_o))^2))
})

c(mean_rmse = mean(outer_rmse), sd_rmse = sd(outer_rmse))

```

Output & Results

A vector of K outer-fold RMSE values; the mean estimates generalisation error and the standard deviation quantifies how much that estimate varies across resampling.

Interpretation

A reporting sentence: “Nested 5×5 cross-validation estimated test RMSE of 1.04 ± 0.08 for the ridge regression with hyperparameter λ tuned on inner folds; single-CV tuning on the same hyperparameter grid would have produced RMSE 0.92 — under-estimating by approximately 12 %, the optimism cost of tuning and evaluating on the same folds.” Always report the outer-fold mean and standard deviation, never a single CV estimate when tuning is involved.

Practical Tips

- Fit *all* preprocessing (imputation, scaling, feature engineering, target encoding) inside each inner fold; preprocessing on the full outer-training set leaks information into validation.
- `tidymodels::nested_cv()` and `mlr3::resampling_nested_cv()` formalise the structure and prevent the manual loop errors that nested CV is famous for.
- For small datasets ($n < 200$), repeated nested CV (repeating the outer split with different random seeds and averaging) reduces the outer-fold variance that small samples introduce.
- The final deployment model is tuned once on the *full* dataset with the chosen hyperparameter; nested CV informs uncertainty about generalisation, not the deployed model itself.
- Parallelise the inner loop with `future` or `doParallel`; the outer loop is typically fast enough to leave serial.
- Computational budget can be substantial: 5×5 nested CV with 5-point hyperparameter grid runs $5 \times 5 \times 5 = 125$ model fits; account for this when planning.

R Packages Used

`rsample` for the outer-loop splits; `tidymodels::tune` and `tune_grid()` for nested-CV-aware hyperparameter optimisation; `mlr3::resampling_nested_cv` and `AutoTuner` for `mlr3`-style nested CV; `caret::train` with `trainControl(method = "repeatedcv")` for an older alternative.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren't.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- `tidymodels` pipelines
- FDR, knockoffs, replication crisis